Theory of Programming Languages

11. Dependent Types

Kihong Heo

KAIST

Motivation

- How can we express and prove the following statements?
  - $\forall x \in \mathbb{N} \,.\, Even(2x)$
  - $\forall P \in \mathbb{N} \to \text{Prop} \,.\, (P(0) \land \forall k \,.\, (P(k) \to P(k+1))) \to \forall n \,.\, P(n)$ (with second-order logic)
- What do we need?
  - Predicates: propositions parameterized by individual objects
  - Quantification over individuals
- First-order logic: reasoning about individual objects
  - *All humans are mortal; Socrates is a human; therefore, Socrates is mortal*

Syntax (1): Terms

- Objects that we are reasoning about
- Terms evaluate to values in an underlying domain (e.g., integers, strings, lists, etc)
- Basic terms: variables (x, y, z, …) and constants (a, b, c, …)
- Composite terms: n-ary functions applied to n terms
  - A constant can be viewed as a 0-ary function
- Example:
  - $a,\ x,\ f(a),\ g(x,\ b),\ f(g(x,\ f(b)))$

Syntax (2): Predicates

- Generalization of propositional variables in PL (p, q, r, …)
- An n-ary predicate takes n terms as arguments
  - A FOL propositional variable is a 0-ary predicate (P, Q, R, …)
- Example:
  - P, $p(f(x), g(x, f(x)))$
  - *isHappy*(x), *love*(x, y), *betterThan*(x, y)

Predicates and Functions

- They look similar but different
- Function terms can be nested within each other and inside relation constants
  - E.g., $f(f(x))$, $p(f(x))$
- Predicates cannot be nested within function terms or other predicates
  - E.g., $f(p(x))$, $p(p(x))$

Syntax (3): Formula

- Atom: basic elements
  - truth symbols ($\bot$ and $\top$), n-ary predicates applied to n terms
- Literal: an atom α or its negation $\neg\alpha$
- Formula: literal, the app. of a logical conn. to formulae, or the app. of a quantifier to a formula

$$
\begin{array}{rcl}
\varphi & \rightarrow & \bot \mid \top \mid p(t_1, \ldots, t_n) \\
& \mid & \neg\varphi \\
& \mid & \varphi_1 \wedge \varphi_2 \\
& \mid & \varphi_1 \vee \varphi_2 \\
& \mid & \varphi_1 \rightarrow \varphi_2 \\
& \mid & \exists x.\varphi \\
& \mid & \forall x.\varphi
\end{array}
$$

Example

- Every dog has its day $\forall x.dog(x) \rightarrow \exists y.day(y) \wedge itsDay(x, y)$
- Some dogs have more days than others $\exists x, y.dog(x) \wedge dog(y) \wedge \#days(x) > \#days(y)$
- The length of one side of a triangle is less than the sum of the lengths of the other two sides

$$\forall x, y, z.triangle(x, y, z) \rightarrow length(x) < length(y) + length(z)$$

- Fermat's Last Theorem

$$\begin{aligned}
&\forall n.integer(n) \wedge n > 2 \\
&\quad \rightarrow \forall x, y, z. \\
&\qquad integer(x) \wedge integer(y) \wedge integer(z) \wedge x > 0 \wedge y > 0 \wedge z > 0 \\
&\qquad\quad \rightarrow x^n + y^n \neq z^n
\end{aligned}$$

First-Order Logic

- Universal quantification

$$\frac{\Gamma \vdash \varphi \qquad x \notin FV(\Gamma)}{\Gamma \vdash \forall x.\varphi}\ \forall \mathsf{I} \qquad\qquad \frac{\Gamma \vdash \forall x.\varphi}{\Gamma \vdash \varphi\{t/x\}}\ \forall \mathsf{E}$$

- Existential quantification

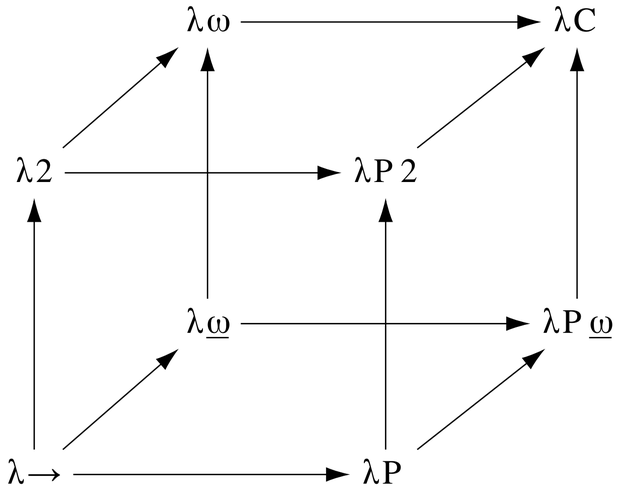
$$\frac{\Gamma \vdash \varphi\{t/x\}}{\Gamma \vdash \exists x.\varphi}\ \exists \mathsf{I} \qquad \frac{\Gamma \vdash \exists x.\varphi \qquad \Gamma \cup \{\varphi\} \vdash \psi \qquad x \notin FV(\Gamma, \psi)}{\Gamma \vdash \psi}\ \exists \mathsf{E}$$

Example

- $(\forall x.human(x) \rightarrow mortal(x)) \rightarrow human(s) \rightarrow mortal(s)$
- $(\exists x.human(x)) \rightarrow (\forall x.human(x) \rightarrow mortal(x)) \rightarrow \exists x.mortal(x)$

Curry-Howard Correspondence

- First-order logic: propositions depend on terms
- Dependent type: types depend on terms
  - E.g., append : $\forall n, m$: nat . int list(n) $\rightarrow$ int list(m) $\rightarrow$ int list($n + m$)
- Recall
  - Simple type ($\lambda \rightarrow$): terms depend on terms
  - Polymorphic type ($\lambda 2$): terms depend on types
  - Type operator ($\lambda \underline{\omega}$): types depend on types
- λP: lambda calculus with dependent types



Example: List

- Dependent list type with type operator τ list(n)
  - Element type τ (i.e., type depending on type)
  - Length n (i.e., type depending on term)
  - (Strictly, $\lambda P\underline{\omega}$ but only consider λP for brevity)
- Example:
  - [1;2;3] : int list(3)
  - [“a”] : string list(1)

Recall: Primitive Recursion for List (1)

- Introduction

$$\frac{}{\Gamma \vdash \texttt{nil} : \tau \text{ list}} \qquad \frac{\Gamma \vdash h : \tau \qquad \Gamma \vdash t : \tau \text{ list}}{\Gamma \vdash h :: t : \tau \text{ list}}$$

- Elimination

$$\frac{\Gamma \vdash E : \tau \text{ list} \qquad \Gamma \vdash E_n : \sigma \qquad \Gamma, x{:}\tau, y{:}\tau \text{ list}, z{:}\sigma \vdash E_c : \sigma}{\Gamma \vdash \texttt{rec}(E, E_n, x.y.z.E_c) : \sigma}$$

Recall: Primitive Recursion for List (2)

- Syntax

$$
\begin{array}{rcl}
E & \to & \ldots \\
 & | & \texttt{nil} \mid E :: E \\
 & | & \texttt{rec}(E, E, x.y.E) \\
K & \to & \ldots \\
 & | & K :: E \mid v :: K \\
 & | & \texttt{rec}(K, E, x.y.E) \\
v & \to & \ldots \mid \texttt{nil} \mid v :: v
\end{array}
$$

- Dynamic semantics

$$\frac{}{\texttt{rec}(\texttt{nil}, E_n, x.y.z.E_c) \to E_n}$$

$$\frac{}{\texttt{rec}(h :: t, E_n, x.y.z.E_c) \to E_c\{h/x\}\{t/y\}\{\texttt{rec}(t, E_n, x.y.z.E_c)/z\}}$$

Type Rules

- Type formation rule for lists of length n

$$\frac{\Gamma \vdash \tau \qquad \Gamma \vdash n : \mathsf{nat}}{\Gamma \vdash \tau\ \mathsf{list}(n)}$$

- Introduction rules

$$\frac{}{\Gamma \vdash \mathtt{nil} : \tau\ \mathsf{list}(\mathtt{0})} \qquad \frac{\Gamma \vdash h : \tau \qquad \Gamma \vdash t : \tau\ \mathsf{list}(n)}{\Gamma \vdash h :: t : \tau\ \mathsf{list}(\mathtt{s}(n))}$$

- Elimination rule

$$\frac{\begin{array}{l}\Gamma \vdash E : \tau\ \mathsf{list}(n) \\ \Gamma \vdash E_z : \sigma(\mathtt{0}, \mathtt{nil}) \\ \Gamma, m : \mathsf{nat}, x : \tau, y : \tau\ \mathsf{list}(m), z : \sigma(m, y) \vdash E_s : \sigma(\mathtt{s}(m), x :: y)\end{array}}{\Gamma \vdash \mathtt{rec}(n, E, E_z, m.x.y.z.E_s) : \sigma(n, E)}$$

Operational Semantics

$$\frac{}{\texttt{rec}(\texttt{0}, \texttt{nil}, E_z, m.x.y.z.E_s) \rightarrow E_z}$$

$$\frac{}{\texttt{rec}(\texttt{s}(n), h :: t, E_z, m.x.y.z.E_s) \rightarrow E_s\{n/m\}\{h/x\}\{t/y\}\{\texttt{rec}(n, t, E_z, m.x.y.z.E_s)/z\}}$$

Example: List Append (1)

- Type of list append? $app : \tau\ \mathsf{list}(n) \to \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(n+m)$
- Where do n and m come from?
- Parameters of dependent function types
  - Corresponding to universally quantified variables under the Curry-Howard correspondence
- List append with dependent type

$$app : \Pi n\!:\!\mathsf{nat}.\ \tau\ \mathsf{list}(n) \to \Pi m\!:\!\mathsf{nat}.\ \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(n+m)$$

$$app = \lambda n\!:\!\mathsf{nat}.\ \lambda l\!:\!\tau\,\mathsf{list}(n).\ \lambda m\!:\!\mathsf{nat}.\ \lambda k\!:\!\tau\ \mathsf{list}(m).\ \mathtt{rec}(n, l, k, r.x.y.z.\ x :: z)$$

Dependent Function Type Π

- Type formation rule

$$\frac{\Gamma \vdash \tau \qquad \Gamma, x : \tau \vdash \sigma(x)}{\Gamma \vdash \Pi x{:}\tau.\ \sigma(x)}$$

- Introduction rules

$$\frac{\Gamma, x{:}\tau \vdash E : \sigma(x)}{\Gamma \vdash \lambda x{:}\tau.E : (\Pi x{:}\tau.\ \sigma(x))}\ \Pi\mathsf{I}$$

- Elimination rule

$$\frac{\Gamma \vdash E_1 : (\Pi x{:}\tau.\sigma(x)) \qquad \Gamma \vdash E_2 : \tau}{\Gamma \vdash E_1\ E_2 : \sigma(E_2)}\ \Pi\mathsf{E}$$

- Operational semantics

$$\frac{}{(\lambda x : \tau.E)\ v \to E\{v/x\}}$$

Example: List Append (2)

- Recall

$$app : \Pi n : \mathsf{nat}.\ \tau\ \mathsf{list}(n) \to \Pi m : \mathsf{nat}.\ \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(n+m)$$

$$app = \lambda n : \mathsf{nat}.\ \lambda l : \tau\ \mathsf{list}(n).\ \lambda m : \mathsf{nat}.\ \lambda k : \tau\ \mathsf{list}(m).\ \mathtt{rec}(n, l, k, r.x.y.z.\ x :: z)$$

- Consider $app\ \mathsf{0}\ \mathtt{nil}\ m\ k$ and $k : \tau\ \mathsf{list}(m)$

$$\begin{array}{lcl}
app\ \mathsf{0} & : & \tau\ \mathsf{list}(\mathsf{0}) \to \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathsf{0}+m) \\
app\ \mathsf{0}\ \mathtt{nil} & : & \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathsf{0}+m) \\
app\ \mathsf{0}\ \mathtt{nil}\ m & : & \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathsf{0}+m) \\
app\ \mathsf{0}\ \mathtt{nil}\ m\ k & : & \tau\ \mathsf{list}(\mathsf{0}+m) \\
 & & \\
k & : & \tau\ \mathsf{list}(m)
\end{array}$$

- Why can these two types be treated as equal?

Type Equivalence

- Definitional equality rules (정의에 따른 동치 규칙)

$$\frac{\Gamma \vdash \tau}{\Gamma \vdash \tau \equiv \tau} \qquad \frac{\Gamma \vdash \tau \equiv \sigma}{\Gamma \vdash \sigma \equiv \tau} \qquad \frac{\Gamma \vdash \tau_1 \equiv \tau_2 \qquad \Gamma \vdash \tau_2 \equiv \tau_3}{\Gamma \vdash \tau_1 \equiv \tau_3}$$

$$\frac{\Gamma \vdash \tau_1 \longrightarrow \tau_2}{\Gamma \vdash \tau_1 \equiv \tau_2} \qquad \frac{\Gamma \vdash n \equiv m : \mathsf{nat}}{\Gamma \vdash \tau\ \mathsf{list}(n) \equiv \tau\ \mathsf{list}(m)}$$

- Definitional equality of nat terms

$$\frac{\Gamma \vdash n : \mathsf{nat}}{\Gamma \vdash 0 + n \equiv n : \mathsf{nat}} \qquad \frac{\Gamma \vdash n : \mathsf{nat} \qquad \Gamma \vdash m : \mathsf{nat}}{\Gamma \vdash s(n) + m \equiv s(n + m) : \mathsf{nat}}$$

(also refl, symm, trans, etc)

- Type conversion

$$\frac{\Gamma \vdash E : \tau \qquad \tau \equiv \sigma \qquad \Gamma \vdash \sigma}{\Gamma \vdash E : \sigma}$$

Example: List Append (3)

- Base case

$$
\begin{array}{lcl}
app\ \mathtt{0} & : & \tau\ \mathsf{list}(\mathtt{0}) \to \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{0}+m) \\
app\ \mathtt{0}\ \mathtt{nil} & : & \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{0}+m) \\
app\ \mathtt{0}\ \mathtt{nil}\ m & : & \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{0}+m) \\
app\ \mathtt{0}\ \mathtt{nil}\ m\ k & : & \tau\ \mathsf{list}(\mathtt{0}+m) \\
app\ \mathtt{0}\ \mathtt{nil}\ m\ k & : & \tau\ \mathsf{list}(m) \\
k & : & \tau\ \mathsf{list}(m)
\end{array}
$$

- Inductive case

$$
\begin{array}{lcl}
app\ \mathtt{s}(n')\ & : & \tau\ \mathsf{list}(\mathtt{s}(n')) \to \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{s}(n')+m) \\
app\ \mathtt{s}(n')\ (x :: l') & : & \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{s}(n')+m) \\
app\ \mathtt{s}(n')\ (x :: l')\ m & : & \tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(\mathtt{s}(n')+m) \\
app\ \mathtt{s}(n')\ (x :: l')\ m\ k & : & \tau\ \mathsf{list}(\mathtt{s}(n')+m) \\
 & & \\
app\ n'\ l'\ m\ k & : & \tau\ \mathsf{list}(n'+m) \\
x :: (app\ n'\ l'\ m\ k) & : & \tau\ \mathsf{list}(\mathtt{s}(n'+m))
\end{array}
$$

- Definitional equality (정의에 따라 같음): by definition of +

$$\tau\ \mathsf{list}(0+m) \equiv \tau\ \mathsf{list}(m) \qquad \tau\ \mathsf{list}(\mathtt{s}(n')+m) \equiv \tau\ \mathsf{list}(\mathtt{s}(n'+m))$$

Example: List Filter

- Type of list filter? $\mathit{filter} : (\tau \to \mathsf{bool}) \to \Pi n : \mathsf{nat}.\tau\ \mathsf{list}(n) \to \tau\ \mathsf{list}(m)$
- Where do m come from?
- Parameter of dependent sum type
  - Corresponding to existentially quantified variables under the Curry-Howard correspondence
- List filter with dependent type

$$
\begin{array}{rcl}
\mathit{filter} & : & (\tau \to \mathsf{bool}) \to \Pi n : \mathsf{nat}.\tau\ \mathsf{list}(n) \to \Sigma m : \mathsf{nat}.\tau\ \mathsf{list}(m) \\
\mathit{filter}\ p\ \mathtt{0}\ \mathtt{nil} & = & \langle \mathtt{0}, \mathtt{nil} \rangle \\
\mathit{filter}\ p\ \mathtt{s}(n')\ (x :: l') & = & \mathtt{let}\ \langle m', k' \rangle = \mathit{filter}\ p\ n'\ l'\ \mathtt{in} \\
 & & \mathtt{if}\ p\ x\ \mathtt{then}\ \langle \mathtt{s}(m'), x :: k' \rangle\ \mathtt{else}\ \langle m', k' \rangle
\end{array}
$$

Dependent Sum Type Σ

- Type formation rule

$$\frac{\Gamma \vdash \tau \qquad \Gamma, x : \tau \vdash \sigma(x)}{\Gamma \vdash \Sigma x{:}\tau.\ \sigma(x)}$$

- Introduction rules

$$\frac{\Gamma \vdash E_1 : \tau \qquad \Gamma \vdash E_2 : \sigma(E_1)}{\Gamma \vdash \langle E_1, E_2 \rangle : (\Sigma x{:}\tau.\ \sigma(x))}\ \Sigma\text{I}$$

- Elimination rule

$$\frac{\Gamma \vdash E_1 : (\Sigma x{:}\tau.\sigma(x)) \qquad \Gamma, x{:}\tau, y{:}\sigma(x) \vdash E_2 : \rho}{\Gamma \vdash (\texttt{let}\ \ \langle x, y \rangle = E_1\ \ \texttt{in}\ \ E_2) : \rho}\ \Sigma\text{E}$$

Example: List Reverse Append

- Consider a reverse-append function

$$rev : \Pi n : \mathsf{nat}.\tau\ \mathsf{list}(n) \to \Pi m : \mathsf{nat}.\tau\ \mathsf{list}(m) \to \tau\ \mathsf{list}(n+m)$$

$$\begin{array}{rcl} rev\ \mathtt{0}\ \mathtt{nil}\ m\ a & = & a \\ rev\ \mathtt{s}(n')\ (x :: l')\ m\ a & = & rev\ n'\ l'\ \mathtt{s}(m)\ (x ::\ a) \end{array}$$

- Type check

$$\begin{array}{rcl} rev\ \mathtt{s}(n')\ (x :: l')\ m\ a & : & \tau\ \mathsf{list}(\mathtt{s}(n') + m) \\ rev\ n'\ l'\ \mathtt{s}(m)\ (x ::\ a) & : & \tau\ \mathsf{list}(n' + \mathtt{s}(m)) \end{array}$$

- Type mismatch, why?
- Proof of $\mathtt{s}(n') + m = n' + \mathtt{s}(m)$ required
  - Propositional equality (증명에 따라 같음)

Summary

- Dependent type: types depend on terms
- First-order logic: propositions depend on terms
- Type checking involves term-level computation
  - Definitional: checked by computation
  - Propositional equality: explicit proofs required
- Foundation of proof assistants (e.g., Lean, Rocq)
  - Constructing proof terms = writing programs
  - Checking proof terms = type checking